SIMATS ENGINEERING ASSESSMENT
TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO3:Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:25
Duration: 1 Hour

CASE STUDY

Code of Conduct:

*I **J.Goutham Kumar Reddy (192311140**)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

J.Goutham Kumar Reddy
Signature of the student:

SIMATS ENGINEERING ASSESSMENT TOOLS

1. **Designing a Grammar for Arithmetic Expressions**

A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as a+b*c.

Case Study Question:

Develop an optimized context-free grammar (CFG) that correctly enforces operator precedence and associativity for the given arithmetic expressions. Explain how your grammar removes ambiguity and supports efficient parsing.

2. **Mini Programming Language Parser Development**

A startup is designing a small scripting language for automating IoT devices. The language supports variable declarations, conditional statements, and loops. However, the parser frequently reports syntax conflicts during compilation.

Case Study Question:

Construct a CFG for the scripting language constructs below and optimize it to eliminate left recursion and reduce parsing conflicts. Demonstrate how the optimized grammar improves parser efficiency.

3. **Grammar Optimization for SQL Query Validation**

A database company is developing a query validation tool for simplified SQL commands such as SELECT, FROM, and WHERE. The original grammar contains redundant productions that increase parsing time.

Case Study Question:

Develop a CFG for simplified SQL query syntax and optimize the grammar by removing unnecessary productions and ambiguity. Discuss how the optimized grammar helps in faster syntax analysis.

4. **Natural Language Processing Chatbot**

A chatbot system processes simple English sentences such as:
Subject + Verb + Object
Example: “Students learn TOC.”
The initial CFG incorrectly accepts grammatically invalid sentences.

Case Study Question:

Design a CFG for the chatbot sentence structure and optimize it to improve language validation accuracy. Explain how the grammar ensures correct sentence formation and rejects invalid inputs.

5. **Parser Design for HTML-like Markup Language**

A web editor company is building a parser for a simplified markup language with opening and closing tags. The existing grammar fails to validate nested tags correctly.

Case Study Question:

Develop and optimize a CFG for validating properly nested tags in the markup language. Show how the grammar supports recursive nesting and improves parsing reliability.

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and wellorganized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING

ASSESSMENT TOOLS

ANSWERS

1. Grammar for Arithmetic Expressions

A naive grammar like:

E -> E + E | E - E | E * E | E / E | (E) | id

is ambiguous. For a + b * c, two different parse trees are possible — one where + is applied first, one where * is applied first — because the grammar gives no rule for precedence or associativity. An ambiguous grammar also slows down parsing (especially for bottom-up parsers), since the parser generator reports shift-reduce conflicts and cannot decide deterministically which rule to reduce by.

Optimized Grammar

We remove ambiguity by **layering the grammar according to precedence** (lowest precedence at the top, highest at the bottom) and by making each level **left-recursive** to enforce **left associativity**:

E -> E + T | E - T | T
T -> T * F | T / F | F F
-> (E) | id

- E (Expression) handles + and - — lowest precedence.
- T (Term) handles * and / — higher precedence.
- F (Factor) handles parentheses and atomic identifiers/numbers — highest precedence.

Why This Removes Ambiguity

1. **Precedence is encoded structurally**: because T can only be reached from E after all +/alternatives are considered, *// productions are always nested *inside* +/- productions in the parse tree, forcing multiplication/division to bind tighter.
2. **Associativity is encoded by recursion direction**: E -> E + T is left-recursive, so a + b + c always parses as (a + b) + c (left-associative), matching standard arithmetic conventions.
3. **Each nonterminal has exactly one unambiguous production path** for any given input — for any sentential form there is only one derivation, hence only one parse tree.

Efficiency Gains

- The grammar is now suitable for **top-down predictive parsing** (after eliminating left recursion, see below) or **LALR(1) bottom-up parsing** (e.g., via YACC/Bison) with **zero shift-reduce conflicts**.
- For recursive-descent parsers, left recursion must be eliminated:

E -> T E'
E' -> + T E' | - T E' | ε
T -> F T'
T' -> * F T' | / F T' | ε
F -> (E) | id

This transformation preserves the language and precedence/associativity while making the grammar **LL(1)-parsable**, enabling single-token lookahead parsing with linear-time performance (O(n) in the length of the input) and no backtracking.

2. Mini Scripting Language for IoT Automation

A language with variable declarations, conditionals, and loops, e.g.:

```
Stmt -> Decl | If | Loop
Decl -> "var" id "=" Expr ";"
If   -> "if" "(" Expr ")" "{" StmtList "}" ElsePart
ElsePart -> "else" "{" StmtList "}" | ε
Loop -> "while" "(" Expr ")" "{" StmtList "}"
```

Conflicts typically arise from:

- **The dangling-else problem** (an else could ambiguously attach to more than one if).
- **Left recursion** in statement/expression lists causing infinite loops in recursive-descent parsers. **Redundant/overlapping productions** across statement types causing FIRST-set collisions.

Optimized CFG

```
Program   -> StmtList
StmtList -> Stmt StmtList | ε
Stmt      -> Decl | Assign | IfStmt | WhileStmt

Decl      -> "var" id "=" Expr ";"
Assign    -> id "=" Expr ";"

IfStmt    -> "if" "(" Expr ")" Block ElseOpt
ElseOpt   -> "else" Block | ε
WhileStmt -> "while" "(" Expr ")" Block
Block     -> "{" StmtList "}"

Expr      -> Expr "||" Term | Term
Term      -> Term "&&" Rel | Rel
Rel       -> Add ("<" | ">" | "==" | "!=") Add | Add
Add       -> Add "+" Mul | Add "-" Mul | Mul
Mul       -> Mul "*" Unary | Mul "/" Unary | Unary
Unary     -> "!" Unary | Add
Atom      -> id | num | "(" Expr ")"
```

Eliminating Left Recursion

Direct left recursion in Add, Mul, Expr, Term (structurally identical to Case 1) is removed using the standard transformation:

```
Add -> Mul Add'
Add' -> "+" Mul Add' | "-" Mul Add' | ε
```

applied at every arithmetic level. StmtList -> Stmt StmtList | ε is already **right-recursive**, so no transformation is needed there — right recursion is safe for recursive descent (though it should be converted to an iterative loop in implementation to avoid deep call stacks).

Resolving the Dangling-Else Conflict

The **"match each** else **with the nearest unmatched `if"** convention is enforced by distinguishing *matched* and *unmatched* statements:

```
Stmt        -> MatchedStmt | UnmatchedStmt
MatchedStmt -> "if" "(" Expr ")" MatchedStmt "else" MatchedStmt
            | OtherStmt
UnmatchedStmt -> "if" "(" Expr ")" Stmt
            | "if" "(" Expr ")" MatchedStmt "else" UnmatchedStmt
```

This guarantees a single derivation for every nested if/else combination, removing the classic shift-reduce conflict that most parser generators (Yacc, ANTLR) otherwise resolve arbitrarily via a default "shift" rule.

Efficiency Gains

- Left-recursion elimination allows a **recursive-descent (LL(1)) parser** — ideal for a lightweight IoT compiler with limited resources.
- Splitting statements by category (Decl, Assign, IfStmt, WhileStmt) means each alternative starts with a distinct keyword or token, so **FIRST sets are disjoint** — the parser can choose a production by looking at only one token of lookahead, avoiding backtracking entirely.
- The dangling-else fix removes the single most common conflict reported by LALR parser generators for C-like languages, so grammar compilation itself is faster and conflict-free.

3. Grammar for Simplified SQL Query Validation

A grammar with excessive or overlapping productions, e.g., separate near-duplicate rules for every clause combination, increases parser states and slows LALR table construction, while also permitting invalid combinations (e.g., WHERE without FROM).

Optimized CFG

```
Query      -> "SELECT" ColumnList "FROM" TableName WhereOpt OrderOpt ";"

ColumnList -> "*" | ColRefList
ColRefList -> id | id "," ColRefList

TableName -> id

WhereOpt   -> "WHERE" Condition | ε
OrderOpt   -> "ORDER" "BY" ColRefList | ε

Condition -> Condition "AND" Comparison
          | Condition "OR" Comparison
          | Comparison
Comparison -> id CompOp Value
CompOp     -> "=" | "!=" | "<" | ">" | "<=" | ">="
Value   -> num | string | id Optimization
```

Steps

1. **Merged near-duplicate productions**: instead of separate rules for SELECT ... FROM ..., SELECT ... FROM ... WHERE ..., SELECT ... FROM ... WHERE ... ORDER BY ..., optional clauses (WhereOpt, OrderOpt) are factored out using **ε-productions**, cutting the number of top-level Query productions from potentially 4+ down to 1.

2. **Left factoring** ColRefList avoids ambiguity between a single column and a comma-separated list — both share a common id prefix, so factoring prevents the parser from needing multi-token lookahead.
3. **Removed unreachable/redundant nonterminals** — a common source of grammar bloat is leftover rules from earlier grammar iterations (e.g., duplicate Condition definitions for WHERE vs. HAVING that were structurally identical); these are unified into a single reusable Condition nonterminal.
4. **Left recursion in** Condition **and** ColRefList is intentional here only if a bottom-up (LALR/SLR) parser is used, since LALR handles left recursion natively and it is *more efficient* than right recursion for such parsers (constant stack depth vs. growing right-recursive stack). If a top-down parser is required instead, these would be converted to right recursion as in Case 1.

Why This Speeds Up Syntax Analysis

- Fewer grammar rules means a **smaller LALR parsing table** (fewer states), which reduces both parser-generation time and runtime memory footprint.
- Enforcing clause order and optionality structurally (WhereOpt, OrderOpt) means **invalid clause orderings are rejected at the grammar level** rather than needing extra semantic checks after parsing — catching errors earlier and cheaper.
- Left-factored ColumnList/ColRefList eliminates backtracking that the original ambiguous grammar would have required to distinguish SELECT * from SELECT col1, col2.

4. CFG for a Simple Subject–Verb–Object Chatbot Grammar

A naive grammar such as:

```
S -> NP VP
NP -> N
VP -> V N
```

accepts sentences with mismatched number agreement or nonsensical word combinations (e.g., mixing singular subjects with plural verbs, or verbs with no valid object), because it treats words as an unstructured token list without categorized lexical rules.

Optimized CFG

```
S    -> NP VP

NP   -> Det N | N | ProperN
VP   -> V NP

Det -> "the" | "a"
N    -> "students" | "student" | "TOC" | "chatbot" | "grammar"
ProperN -> "Alice" | "Bob"
V    -> "learn" | "learns" | "study" | "studies"
```

To enforce **subject–verb agreement** (a common source of "grammatically invalid but CFG-accepted" sentences), the grammar is refined using **feature-annotated nonterminals** (a standard CFG technique to simulate agreement without full context-sensitivity):

```
S    -> NP_sg VP_sg | NP_pl VP_pl

NP_sg -> Det N_sg | ProperN
NP_pl -> Det N_pl | N_pl
VP_sg -> V_sg NP
VP_pl -> V_pl NP

N_sg -> "student" | "chatbot"
N_pl -> "students" | "chatbots"
V_sg -> "learns" | "studies" V_pl
-> "learn" | "study"

NP   -> NP_sg | NP_pl
```

How This Ensures Correct Sentence Formation

1. **Structural separation by grammatical category** (Det, N, V, ProperN) means the grammar can only combine words in linguistically valid roles — a verb can never appear where a noun is expected, and vice versa.
2. **Number agreement via nonterminal splitting** (_sg / _pl) forces the derivation of S to pick *matching* singular or plural branches for subject and verb — "Students learns" cannot be derived because NP_pl only combines with VP_pl, not VP_sg.
3. Invalid inputs (e.g., "TOC learn students" — object-subject-verb order) are rejected because **word order is fixed by the production** S -> NP VP **and** VP -> V NP, disallowing any other ordering.

Efficiency/Accuracy Trade-off

This grammar remains context-free (agreement is encoded via a bounded, finite feature — number — expanded into separate nonterminals) so it is still efficiently parsable in linear time by a standard CYK or Earley parser, while substantially reducing false positives compared to the unannotated version. For richer agreement (tense, person, gender), the same pattern scales but grammar size grows multiplicatively with each feature — beyond a certain point, augmenting with a lightweight semantic/agreement checker as a post-parse pass is more practical than an ever-larger CFG.

5. CFG for a Simplified HTML-like Markup Language

A flat grammar like:

```
Doc -> Tag*
Tag -> "<" id ">" Content "</" id ">"
```

without recursive structure fails to validate that closing tags match their corresponding opening tags, and cannot correctly represent arbitrarily deep nesting (e.g.,) — it may accept mismatched or improperly nested tags like .

Optimized CFG

```
Document -> ElementList

ElementList -> Element ElementList | ε

Element   -> OpenTag ElementList CloseTag
          | SelfClosingTag
          | Text

OpenTag   -> "<" TagName Attrs ">"
CloseTag -> "</" TagName ">"
SelfClosingTag -> "<" TagName Attrs "/>"

TagName   -> id
Attrs     -> Attr Attrs | ε
Attr      -> id "=" string

Text      -> textchar Text | textchar
```

A key refinement (matching open/close tag names, which pure CFGs cannot express directly since CFGs have no memory of *which* identifier was used) is typically handled one of two ways:

Option A — Grammar-per-tag-name (parameterized/context-sensitive extension): For a small, fixed, known set of tag names, define matched pairs explicitly:

```
Element -> "<div>" ElementList "</div>"
         | "<span>" ElementList "</span>"
         | "<p>" ElementList "</p>"
         | Text
```

This keeps it a true CFG and guarantees matching by construction, at the cost of one production pair per tag name.

Option B — CFG + semantic action (standard real-world approach): Keep the general TagName-based grammar above (which is context-free and handles arbitrary tag names), and enforce the "open name = close name" constraint with a **semantic check during/after parsing** (e.g., a tag-name stack maintained by the parser) — this is how real HTML/XML parsers work, since **tag-name matching is not a context-free property** (it requires unbounded memory of *which* name was opened, which needs a stack keyed by content, technically making the *full* language context-sensitive, though a pushdown automaton equivalent to the parser's own stack handles it naturally).

How This Supports Recursive Nesting

- Element -> OpenTag ElementList CloseTag is **self-referential through** ElementList, so an Element can contain any number of nested Elements, each of which can itself contain nested Elements — this directly models arbitrary-depth nesting (<a><c></c>), which a flat, non-recursive grammar cannot express.
- Because the grammar is a natural **context-free (in fact, near-Dyck-language) structure**, it maps directly onto a **pushdown automaton**: each OpenTag pushes onto the stack, each CloseTag pops, and a well-formed document corresponds exactly to a balanced stack at the end of parsing. This is the same

principle used for matching parentheses, guaranteeing that improperly nested or unmatched tags are provably rejected.

Efficiency Gains

- Recursive-descent or LL(1) parsing works directly on this grammar since OpenTag/CloseTag/Text/SelfClosingTag alternatives are distinguished by their first token (<tag>, </tag>, <tag/>, or plain text), giving disjoint FIRST sets and requiring no backtracking.
- Using a parser-maintained stack for tag-name matching (Option B) keeps parsing **linear time, O(n)**, in the size of the document — the same complexity class as well-known real-world HTML/XML parsers — while still catching all mismatched-nesting errors deterministically.